

# Feature Engineered Approach to Classifying Questions-Answers Text

In partial completion of the CSE440: Natural Language Processing Course in BRAC University

Mohammad Jabir Safa Khandoker

*Department of CSE*

*BRAC University*

Dhaka, Bangladesh

jabir.safa.khandoker@g.bracu.ac.bd

Md. Saadat Rahman

*Department of CSE*

*BRAC University*

Dhaka, Bangladesh

md.saadat.rahman@g.bracu.ac.bd

Nafiur Rahman Afnan

*Department of CSE*

*BRAC University*

Dhaka, Bangladesh

nafiur.rahman.afnan@g.bracu.ac.bd

**Abstract**—This project compares TF-IDF and GloVe word embeddings for multi-class text classification of question-answer documents. We preprocess text by removing HTML tags and applying NLTK normalization (lowercasing, tokenization, stop-word removal, and lemmatization). Two input formulations are evaluated: (i) cleaned QA text and (ii) feature-engineered text combining question title, content, and best answer. We train Logistic Regression and neural architectures including Deep Neural Networks, SimpleRNN, GRU, LSTM, and their bidirectional variants, with hyperparameter tuning for selected models. Performance is evaluated using accuracy, macro F1-score, confusion matrices, and classification reports on a held-out test set. Results demonstrate the trade-offs between representation methods, model complexity, and classification performance for QA topic categorization.

**Index Terms**—TF-IDF, GloVe, RNN, GRU, LSTM, Bidirectional, HTML tags, Text Classification, Feature Engineering, Logistic Regression

## I. INTRODUCTION

Text classification is a fundamental task in Natural Language Processing (NLP) that involves automatically categorizing text documents into predefined classes. With the exponential growth of digital text data, efficient and accurate text classification systems have become crucial for various applications, including sentiment analysis, spam detection, topic categorization, and information retrieval [1].

The effectiveness of text classification systems largely depends on two key components: the word representation technique used to convert text into numerical features and the machine learning model employed for classification. Traditional approaches rely on statistical methods such as Term Frequency-Inverse Document Frequency (TF-IDF), while modern approaches leverage distributed word representations like GloVe [2] embeddings.

Recent advances in deep learning have introduced sophisticated neural architectures for text classification, including Deep Neural Networks (DNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs) [3], and Gated Recurrent Units (GRUs) [4]. These models have demonstrated superior performance in capturing sequential dependencies and semantic relationships in text data.

## II. DATASET DESCRIPTION

We use the Question Answer Classification Dataset, which contains text samples from 10 domains:

- |                          |                            |
|--------------------------|----------------------------|
| i. Science & Mathematics | vi. Health                 |
| ii. Business & Finance   | vii. Entertainment & Music |
| iii. Society & Culture   | viii. Computers & Internet |
| iv. Sports               | ix. Politics & Government  |
| v. Education & Reference | x. Family & Relationships  |

The dataset is split into training (93,333 samples) and test (59,999 samples) sets with a balanced distribution across classes.

Each sample consists of structured QA text with HTML formatting, including Question Title, Question Content, and Best Answer sections.

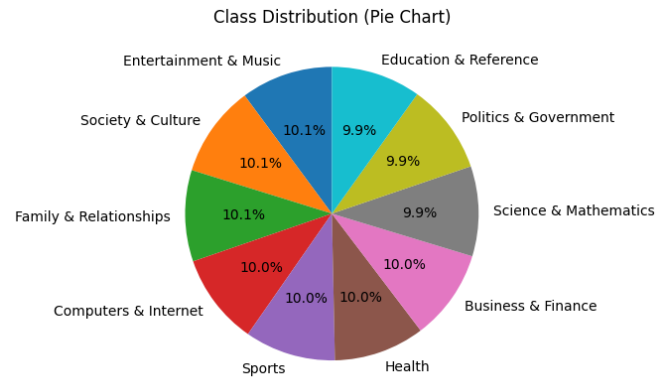


Fig. 1. Class Distribution (Pie Chart)

## III. EXPLORATORY DATA ANALYSIS (EDA)

We performed comprehensive EDA to understand the dataset characteristics:

- Class Distribution:** Verified balanced distribution across 10 classes (~9,000-9,500 samples per class)
- Text Length Analysis:** Analyzed character and word count distributions to determine appropriate sequence lengths for neural networks

3. **Word Cloud Visualization:** Generated word clouds per class to identify distinctive keywords
4. **Data Quality Checks:** Identified HTML tags in 100% of samples, confirming the need for preprocessing

**Key Findings:** - Average text length: ~80-100 words per sample - All samples contain HTML tags requiring removal - Classes are well-balanced, suitable for standard evaluation metrics - Distinctive keywords per class visible in word clouds

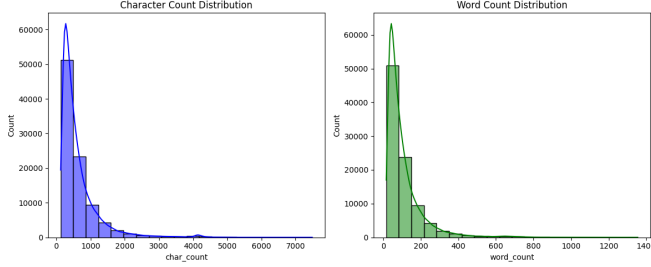


Fig. 2. Word and Character Count by Class

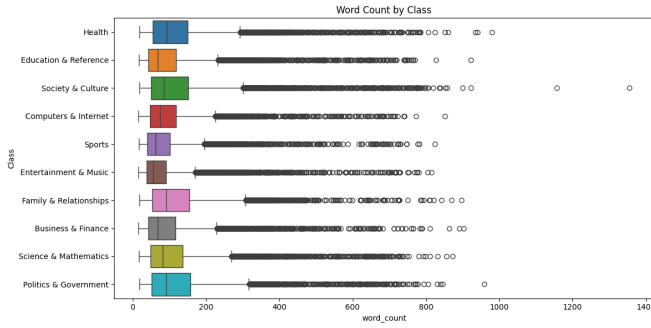


Fig. 3. Box Plot Distribution by Class



Fig. 4. Word Cloud by Class

## IV. DATASET PRE-PROCESSING

We implemented a comprehensive pre-processing pipeline:

### A. HTML Tags Removal

- Detected HTML tags in all samples using regex pattern matching (`<[>]+>`)
- Removed tags while preserving text content
- Result: Cleaned text without markup noise

### B. Feature Engineering

- Extracted structured components using regex:
  - Question Title
  - Question Content
  - Best Answer
- This enabled feature engineering by combining specialized token phrases
- **Standard Version:** Cleaned QA text with HTML removed
- **Feature-Engineered Version:** Combined Title + Content + Answer without headers and later on added token phrases

Both versions were evaluated to compare pre-processing strategies.

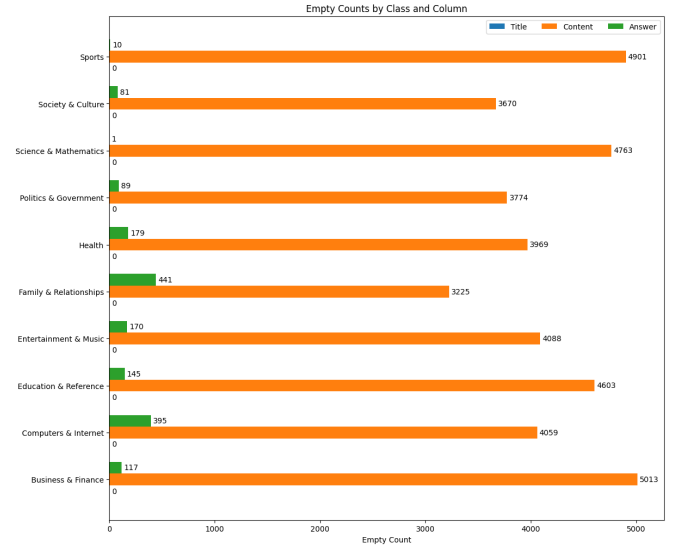


Fig. 5. Empty Column Summary by Class

### C. NLTK

Applied systematic text cleaning in the following order:

- **Lowercasing:** Convert all text to lowercase for consistency
- **URL/Email Removal:** Remove URLs and email addresses using regex patterns
- **Non-alphabetic Removal:** Keep only letters and spaces (remove numbers, punctuation)
- **Tokenization:** Split text into individual words using NLTK tokenizer
- **Stopword Removal:** Remove common English stopwords (the, a, is, etc.)
- **Lemmatization:** Convert words to root form using WordNetLemmatizer (running → run, better → good)

## V. POST PREPROCESSING EDA

After removing the HTML tags and lemmatizing the words through pre-processing, we can see a drastic change in Word Cloud structures.



Fig. 6. Word Cloud by Class (Standard Data)



Fig. 7. Word Cloud by Class (Feature Engineered Data)

## VI. ENCODING

### A. Label Encoding

- Converted text labels to integers (0-9) using LabelEncoder
- Fit encoder only on training data to prevent data leakage
- Applied same mapping to validation and test sets
- Example: “Health”  $\rightarrow$  5, “Sports”  $\rightarrow$  9

### B. One Hot Encoding

- Converted integer labels to binary vectors for neural networks
- Required for categorical\_crossentropy loss function
- Shape: (n\_samples, 10) for 10 classes
- Example: Class 5  $\rightarrow$  [0,0,0,0,0,1,0,0,0,0]

In order to reduce code duplication and complexity, we functionalized all the codes and stored the data in dictionary format as shown in Table: I:

## VII. MODEL TRAINING

This section enumerates the hyperparameters explicitly set in code for each model used in the project.

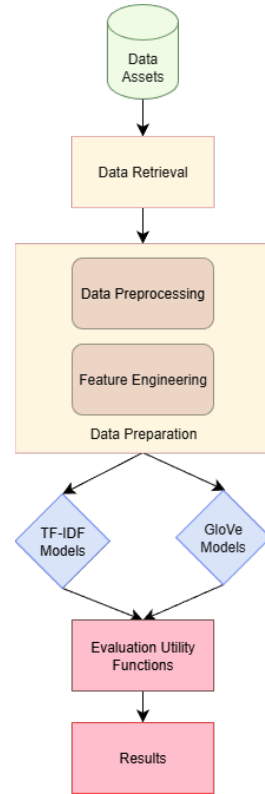


Fig. 8. Model Training Pipeline

### A. TF-IDF Representation

The most specific `TfidfVectorizer` configuration used in the project is summarized below:

- **dtype:** `np.float32` — reduces memory footprint for sparse matrices.
- **max\_df:** `0.9` — removes extremely common, low-information terms.
- **max\_features:** `30000` — limits dimensionality for speed and memory efficiency.
- **min\_df:** `5` — removes very rare terms that add noise.
- **ngram\_range:** `(1, 2)` — incorporates bigrams to capture short phrases.
- **stop\_words:** `english` — removes common function words.

**1) Logistic Regression:**

a) *Base:*

- `random_state`: 42 — ensures reproducibility.
- `max_iter`: 1000 — increased to guarantee convergence.

*b) Hyperparameter:*

- C: 0.25 — inverse regularization strength, tuned to reduce overfitting in high-dimensional TF-IDF space.
- penalty: L2 — standard and stable regularization for sparse text features.

TABLE I  
DATASET SPLITS AND ENCODING TYPES

| Data Type          | Encoding         | X_train                 | y_train                     | X_val                 | y_val                     | X_test                 | y_test                     |
|--------------------|------------------|-------------------------|-----------------------------|-----------------------|---------------------------|------------------------|----------------------------|
| Standard Data      | Label Encoding   | data_std<br>['X_train'] | data_std<br>['y_train_enc'] | data_std<br>['X_val'] | data_std<br>['y_val_enc'] | data_std<br>['X_test'] | data_std<br>['y_test_enc'] |
| Standard Data      | One-Hot Encoding | data_std<br>['X_train'] | data_std<br>['y_train_oh']  | data_std<br>['X_val'] | data_std<br>['y_val_oh']  | data_std<br>['X_test'] | data_std<br>['y_test_oh']  |
| Feature Engineered | Label Encoding   | data_eng<br>['X_train'] | data_eng<br>['y_train_enc'] | data_eng<br>['X_val'] | data_eng<br>['y_val_enc'] | data_eng<br>['X_test'] | data_eng<br>['y_test_enc'] |
| Feature Engineered | One-Hot Encoding | data_eng<br>['X_train'] | data_eng<br>['y_train_oh']  | data_eng<br>['X_val'] | data_eng<br>['y_val_oh']  | data_eng<br>['X_test'] | data_eng<br>['y_test_oh']  |

## 2) Deep Neural Network (DNN):

### a) Base:

- Dense layers: [256, 128, 10]
- Dropout rates: [0.25, 0.5]
- Activation: [relu, relu, softmax]

### b) Hyperparameter:

- Dense layers: [128, 64, 32, 10]
- Dropout rates: [0.25, 0.5, 0.3]
- Activation: [relu, relu, tanh, softmax]

## B. Skipgram (GloVe Embeddings)

All neural models were trained using the following global decisions:

- **Embedding Matrix:** Using a pretrained embedding matrix dataset from Kaggle under the name glove.6B.100d.txt that contains 6 Billion words with 100 dimensions each.
- **Embedding dimension:** EMBEDDING\_DIM = 100 — matches the selected pretrained embedding dimensionality; 100 dimensions provide a strong tradeoff between performance and computational cost.
- **Input Length:** Calculating the arithmetic mean of all the text of Standard Data (MAX\_SEQUENCE\_LENGTH\_STD = 78) and Feature Engineered Data (MAX\_SEQUENCE\_LENGTH\_ENG = 70) and averaging them  $(78+70)/2 = 74$ , we get a general input length (MAX\_SEQUENCE\_LENGTH) for all the the Neural Network experiments. We then pad shorter sequences with zeros at the end or truncate longer sequences from the end to reach the fixed length of MAX\_SEQUENCE\_LENGTH.

All neural models were compiled and evaluated using the following shared setup in the run\_nn\_experiment function:

- **Optimizer:** Adam with learning rate 0.001
- **Loss function:** Categorical cross-entropy
- **Metric:** Accuracy
- **Epochs:** 1000 (with early stopping)
- **Batch size:** 32
- **Early stopping:** Patience of 10 epochs monitoring val\_loss with best-weight restoration

## 1) Embedded Deep Neural Network (DNN):

### a) Base:

- Dense layers: [256, 128, 10]
- Dropout rates: [0.15, 0.25]
- Activation: [relu, tanh, softmax]

### b) Hyperparameter:

- Dense layers: [128, 64, 32, 10]
- Dropout rates: [0.15, 0.2, 0.25]
- Activation: [relu, relu, tanh, softmax]

## 2) Simple Recurrent Neural Networks (RNN):

### a) Base:

- Units: [64]
- Dense layers: [10]
- Dropout rates: [0.3]
- Activation: [tanh, softmax]

### b) Hyperparameter:

- Units: [128, 64]
- Dense layers: [10]
- Dropout rates: [0.25, 0.3]
- Activation: [tanh, tanh, softmax]
- Stack: return\_sequences=True

## 3) Gated Recurrent Unit (GRU):

### a) Base:

- Units: [64]
- Dense layers: [10]
- Dropout rates: [0.3]
- Activation: [softmax]

### b) Hyperparameter:

- Units: [128, 64]
- Dense layers: [32, 10]
- Dropout rates: [0.4, 0.4]
- Activation: [relu, softmax]
- Stack: return\_sequences=True

## 4) Long Short Term Memory (LSTM):

### a) Base:

- Units: [64]
- Dense layers: [10]
- Dropout rates: [0.3]
- Activation: [softmax]

b) *Hyperparameter:*

- Units: [128, 64]
- Dense layers: [10]
- Dropout rates: [0.4, 0.4]
- Activation: [softmax]
- Stack: return\_sequences=True

5) **Bidirectional RNN:**

a) *Base:*

- Units: [64]
- Dense layers: [10]
- Dropout rates: [0.3]
- Activation: [softmax]

b) *Hyperparameter:*

- Units: [128, 64]
- Dense layers: [10]
- Dropout rates: [0.4, 0.4]
- Activation: [softmax]
- Stack: return\_sequences=True

6) **Bidirectional GRU:**

a) *Base:*

- Units: [64]
- Dense layers: [10]
- Dropout rates: [0.3]
- Activation: [softmax]

b) *Hyperparameter:*

- Units: [128, 64]
- Dense layers: [64, 10]
- Dropout rates: [0.4, 0.4]
- Activation: [relu, softmax]
- Stack: return\_sequences=True

7) **Bidirectional LSTM:**

a) *Base:*

- Units: [64]
- Dense layers: [10]
- Dropout rates: [0.3]
- Activation: [softmax]

b) *Hyperparameter:*

- Units: [128, 64]
- Dense layers: [10]
- Dropout rates: [0.4, 0.4]
- Activation: [softmax]
- Stack: return\_sequences=True

## VIII. MODEL COMPARISON

### A. Overall Model Comparison

1) *Best Performing Model:* Hyperparameter tuned Bidirectional LSTM model with GloVe Embeddings run on Feature Engineered Data.

- **Accuracy:** 0.7017
- **F1 Macro:** 0.6992

2) *Worst Performing Model:* Hyperparameter tuned RNN model with GloVe Embeddings run on Standard Data.

- **Accuracy:** 0.4245
- **F1 Macro:** 0.3959

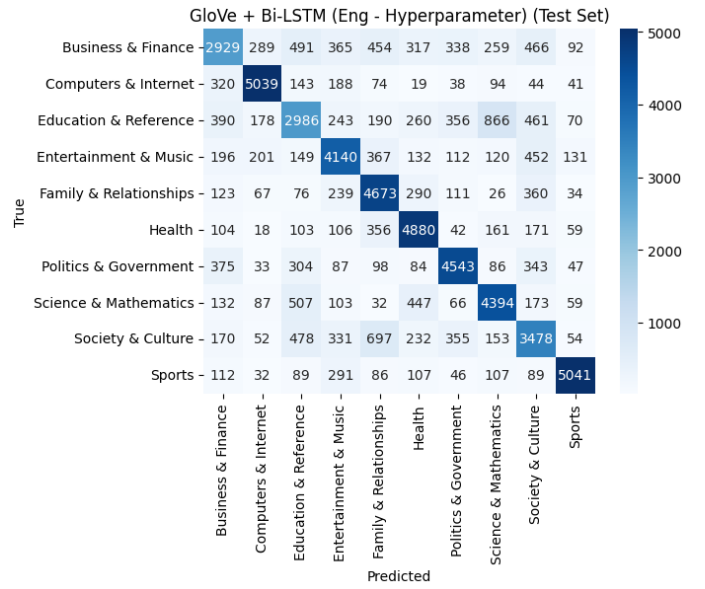


Fig. 9. Overall Best Performing Model

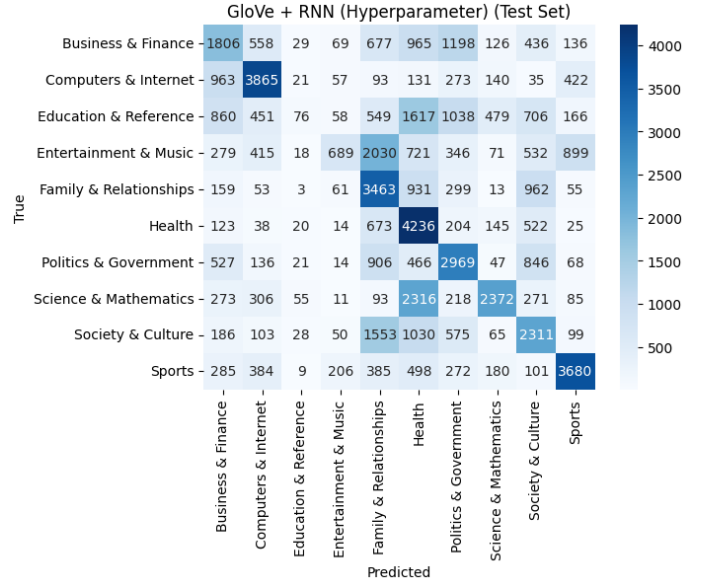


Fig. 10. Overall Worst Performing Model

### B. Best Models Comparison

1) *Best Machine Learning Model:* Base Logistic Regression Model with TF-IDF Vectors run on Standard Data

- **Accuracy:** 0.6834
- **F1 Macro:** 0.6817

2) *Best Neural Network Model:* \*See The Overall Best Model Above Figure: 9

## IX. CONCLUSION

This project evaluated various word representation techniques and neural network architectures for multi-class classification on a balanced dataset of question-answer pairs. Our

TABLE II  
PERFORMANCE COMPARISON OF MODELS ON STANDARD AND FEATURE-ENGINEERED DATA

| Model                       | Standard Data |               |        | Feature Engineered |               |               |
|-----------------------------|---------------|---------------|--------|--------------------|---------------|---------------|
|                             | Acc.          | F1            | AUC    | Acc.               | F1            | AUC           |
| TF-IDF + LR                 | <b>0.6834</b> | <b>0.6817</b> | 0.9383 | 0.6830             | 0.6814        | 0.9380        |
| TF-IDF + LR (HPT)           | 0.6751        | 0.6725        | 0.9339 | 0.6742             | 0.6716        | 0.9339        |
| TF-IDF + DNN                | 0.6593        | 0.6527        | –      | 0.6577             | 0.6526        | –             |
| TF-IDF + DNN (HPT)          | 0.6549        | 0.6496        | –      | 0.6571             | 0.6524        | –             |
| GloVe + DNN                 | 0.5710        | 0.5632        | 0.8994 | 0.5815             | 0.5748        | 0.9049        |
| GloVe + DNN (HPT)           | 0.5751        | 0.5704        | 0.9022 | 0.5839             | 0.5774        | 0.9053        |
| GloVe + RNN                 | 0.5129        | 0.4983        | 0.8693 | 0.5103             | 0.5001        | 0.8627        |
| GloVe + RNN (HPT)           | 0.4245        | 0.3959        | 0.8237 | 0.4608             | 0.4386        | 0.8416        |
| GloVe + GRU                 | 0.6987        | 0.6934        | 0.9485 | 0.6986             | 0.6957        | 0.9480        |
| GloVe + GRU (HPT)           | <b>0.7037</b> | 0.6969        | 0.9489 | 0.7000             | 0.6945        | 0.9482        |
| GloVe + LSTM                | 0.6963        | 0.6920        | 0.9460 | 0.6952             | 0.6907        | 0.9461        |
| GloVe + LSTM (HPT)          | 0.7003        | 0.6956        | 0.9477 | 0.7033             | 0.6984        | 0.9481        |
| GloVe + BiRNN               | 0.6393        | 0.6345        | 0.9190 | 0.6372             | 0.6340        | 0.9192        |
| GloVe + BiRNN (HPT)         | 0.6145        | 0.6059        | 0.9050 | 0.6343             | 0.6277        | 0.9159        |
| GloVe + BiGRU               | 0.6956        | 0.6901        | 0.9473 | 0.6963             | 0.6916        | 0.9477        |
| GloVe + BiGRU (HPT)         | 0.7009        | 0.6944        | 0.9483 | 0.7012             | 0.6940        | 0.9487        |
| GloVe + BiLSTM              | 0.6970        | 0.6922        | 0.9479 | 0.6985             | 0.6935        | 0.9481        |
| <b>GloVe + BiLSTM (HPT)</b> | 0.7005        | 0.6980        | 0.9488 | 0.7017             | <b>0.6992</b> | <b>0.9493</b> |

Note: Acc. - Accuracy; F1 - F1 Macro; AUC - Area Under Curve; HPT - Hyperparameter Tuned.

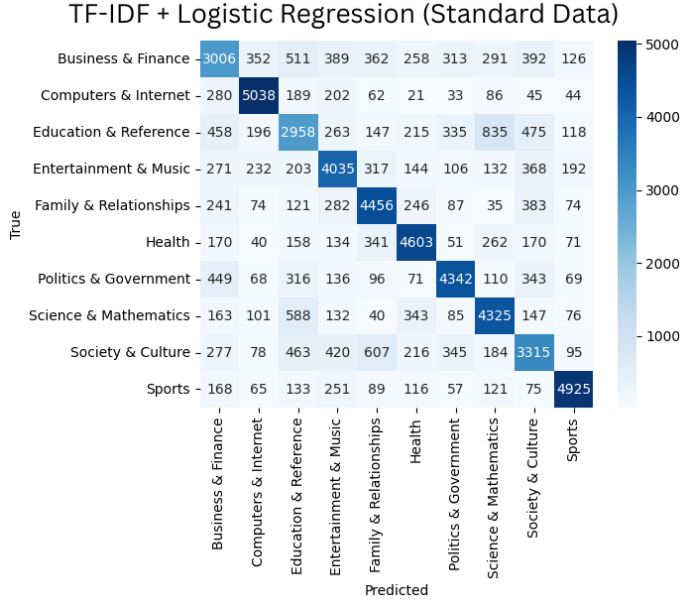


Fig. 11. Best Performing Machine Learning Model

findings indicate that gated recurrent architectures, specifically LSTM and GRU models utilizing GloVe embeddings, significantly outperformed traditional SimpleRNN structures in capturing the semantic nuances of the 10 domain classes. Notably, the **Hyperparameter tuned Bidirectional LSTM model on Feature Engineered data** achieved the highest overall performance with an accuracy of **0.7017** and a macro F1-score of **0.6992**. In contrast, **simple RNN** variants struggled

with sequence dependencies, as evidenced by the worst performance of **0.4245** accuracy on standard data. Furthermore, feature engineering, which combined question titles, content, and best answers without these headers and appropriate empty indicator tokens, provided a measurable boost for deep learning architectures compared to cleaned text alone.

#### ACKNOWLEDGMENT

The authors would like to express their gratitude for the computational resources and tools that made this research possible. Special thanks to Google Gemini for providing technical insights during the development phase and to Md. Saadat Rahman for providing the hardware support via an NVIDIA RTX 4070 GPU, which facilitated the efficient training of the complex bidirectional neural models.

#### REFERENCES

- [1] C. C. Aggarwal and C. Zhai, "A survey of text classification algorithms," in *Mining Text Data*. Springer, 2012, pp. 163–222.
- [2] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2014, pp. 1532–1543.
- [3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [4] K. Cho, B. van Merriënboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.